CubeNN: A Rubik's Cube-Inspired Neural Architecture with Block-Sparse Attention and Cascade Inference

/titleCubeNN: A Rubik's Cube-Inspired Neural Architecture // with Hierarchical Block-Sparse Attention and Cascade Inference

/maketitle

/beginabstract
We present CubeNN, a novel neural network architecture that repurposes Rubik's cube geometry as a computational substrate. Unlike Transformers that operate on flat sequences with quadratic attention, CubeNN organizes neurons across 14 faces of a cube—6 standard faces and 8 diagonal ``X-faces''—and performs block-sparse hierarchical attention with $O(B^2 \log N)$ complexity. A tree-structured cascade explores the state space in parallel, selecting the optimal path for chain-based refinement. Implemented entirely on GPU with zero CPU synchronization during inference, CubeNN achieves 165 inferences per second at $N=32$ on an [illegible] would [illegible] introduces three innovations: (1) geometric attention via face-block decomposition with diagonal cross-face pathways, (2) pyramid upsampling for multi-resolution feature refinement, and (3) a hybrid tree-chain cascade enabling parallel

Introduction

The Transformer vaswani2017attention revolutionized deep learning through self-attention, but its $O(L^2)$ complexity limits scalability. Sparse attention variants child2019generating,beltagy2020longformer,zaheer2020bigbird reduce this to $O(L \log L)$ or $O(L \sqrt{L})$, and vision architectures liu2021swin,dosovitskiy2020vit apply window-based attention to 2D grids. However, all these approaches operate on flat or 2D token sequences, missing the opportunity to exploit 3D geometric structure as an inductive bias.

We propose CubeNN, which organizes computation on the faces of a cube. The central insight is that a Rubik's cube provides a natural topology for neural computation: each face hosts a grid of neurons, block-sparse attention operates within and across faces, and diagonal ``X-faces'' enable efficient cross-face communication without quadratic overhead. The architecture draws inspiration from the human fovea—coarse global perception combined with fine local detail—implemented through a two-level block-sparse attention mechanism.

Beyond the static architecture, CubeNN introduces a cube cascade: a hybrid tree-chain inference strategy where multiple cube states are explored in parallel (tree phase), and the optimal state is refined to arbitrary depth (chain phase). This enables a form of learned test-time computation with guaranteed convergence.

Architecture

The 14-Face Cube

CubeNN's fundamental computational unit is a cube with 14 faces:

/beginitemize
/item 6 standard faces (front, back, left, right, top, bottom): Each face is an $N \times N$ grid of neurons. All 6 faces participate in block-sparse attention and feed-forward computation.
/item 8 diagonal X-faces: Formed by the body diagonals of the cube, each X-face provides a cross-face communication pathway. Unlike standard faces, X-faces do not perform attention; they serve as information bridges, projecting features across faces, upsampling to higher resolutions, and fusing back.
/enditemize

The cube topology provides a natural graph structure: each standard face connects to 4 X-faces, and each X-face connects to 3 standard faces. This bipartite connectivity ensures that information propagating through X-faces reaches all other faces in at most 2 hops.

Block-Sparse Hierarchical Attention

For a face with N^2 neurons, full self-attention requires $O(N^4)$ operations per face—prohibitively expensive. CubeNN uses a two-level block-sparse attention:

Coarse level: The face grid is divided into $B \times B$ blocks. Each block computes a summary vector (mean pool). Global attention operates between all B^2 block summaries, providing a holistic receptive field at $O(B^4)$ cost.

Fine level: Within each block, neurons attend only to other neurons in the same block. This local attention captures fine-grained relationships at $O(N^2 \cdot (N/B)^2)$ cost.

The coarse and fine attention outputs are fused via a learnable mixing coefficient α:

$$\text{output} = \alpha \cdot \text{fine} + (1-\alpha) \cdot \text{coarse}$$

Total attention complexity: $O(B^4 + N^2 \cdot (N/B)^2) \approx O(N^2 \log N)$ when $B = \sqrt{N}$.

Attention FLOP comparison vs. full attention.

N	Full Attention	CubeNN (block-sparse)	Reduction
32	201.3M	0.37M	544$\times$
64	3.2G	2.2M	1,454$\times$
128	137G	13M	10,300$\times$
256	2.27T	52M	42,300$\times$

X-Face Communication Cycle

The X-face cycle enables cross-face information flow without per-face attention:

1. Project: Each X-face computes a weighted sum of its 3 adjacent standard faces.
2. Upsample: X-faces are upsampled via bilinear interpolation to higher resolutions (e.g., $32 \rightarrow 64 \rightarrow 128$), refined with a learnable 3×3 convolution, then fused back to standard faces.
3. Repeat: The project $\rightarrow$ upsample $\rightarrow$ fuse cycle is repeated, enabling iterative refinement with expanding receptive fields.

Cube Cascade: Tree + Chain Inference

CubeNN's inference strategy combines breadth-first exploration with depth-first refinement:

Tree Phase (Exploration): Starting from the root cube, each cube spawns up to K child cubes initialized from X-face overshoot residuals. The tree grows breadth-first to depth D, with convergence detected by comparing parent-child output similarity (L2 distance $< \varepsilon$). A tree of depth 2 with branching factor 8 produces 23 cubes, exploring diverse computational paths in parallel.

Chain Phase (Refinement): The leaf node with maximum output variance is selected for infinite-depth refinement. Each refinement step applies one full cube forward pass. Convergence is detected when the variance delta between steps falls below threshold, typically within 3--5 iterations.

Conclusion

DeepCubeA: mcAleer2019deepcube uses neural networks to solve the Rubik's cube puzzle. CubeNN inverts this relationship: the cube itself becomes the neural architecture.

Geometric Deep Learning: Bronstein et al. bronstein2021geometric provide a theoretical framework for learning on non-Euclidean domains. CubeNN instantiates these ideas on a cube manifold with explicit computational kernels.

Related Work

Sparse Transformers: Sparse Transformer child2019generating uses fixed sparse patterns; Longformer beltagy2020longformer combines sliding window with global attention; BigBird zaheer2020bigbird adds random attention. All operate on 1D sequences. CubeNN's block-sparse attention is most similar to Swin Transformer liu2021swin, which uses shifted windows on 2D grids. CubeNN extends this to 3D with diagonal cross-face connections.

Tree-Cascade Effectiveness

At $N=32$, the tree phase explores 73 cubes (depth-2, branch-8) in 386ms GPU time. Output variance increases from 1.07 (root) to 1.11 (best leaf), demonstrating that parallel exploration discovers higher-quality states. The subsequent chain phase converges in 3 steps (70ms), refining the best state by an additional 6.7% in variance.

The sub-quadratic complexity is empirically verified: scaling from $N=32$ to $N=128$ (16$\times$ more neurons) increases per-cube time by only 20$\times$, compared to the 256$\times$ increase that full attention would require.

Scaling Analysis

Experiments

/begintable[h]
/centering
/captionCubeNN scaling on AMD Radeon RX Vega 64 (14nm, 12.6 TFLOPS FP32).
/begintabular|rrrrr
/toprule
N & Neurons/Cube & Cubes & Time (FP32) & Time (FP16) & ms/cube //
/midrule
32 & 6,144 & 73 & 428ms & 425ms & 5.8 //
64 & 24,576 & 21 & 1023ms & 813ms & 39 //
128 & 98,304 & 1 & -- & 115ms & 115 //
/bottomrule
/endtabular
/endtable

GPU Implementation

CubeNN is implemented in Metal for AMD GPUs with three key optimizations:
/beginenumerate
/item Single encoder, single commit: All 10,658 kernel dispatches for 73 cubes are encoded into one command buffer, eliminating CPU-GPU synchronization overhead. Pre-cached pipeline states reduce encoding time from 1022ms to 42ms.
/item Buffer offsets: All cube face data resides in one contiguous pool buffer. Kernels accept a face/_offset parameter (buffer index 15), avoiding per-face buffer rebinding or blit copies.
/item FP16 support: Mixed-precision kernels use half for data storage with float accumulators. FP16 provides

CubeNN demonstrates that geometric constraints—when designed as computational primitives rather than data representations—can yield architectures with fundamentally different scaling properties. The block-sparse attention achieves 300–10,000× reduction in FLOPs compared to full attention while maintaining global ... cube cascade enables a form of test time

Future work includes: (1) training CubeNN on 3D perception tasks where its geometric bias is most natural, (2) exploring deeper cascades (SD > 35) with learned branching policies, and (3) porting to Apple Silicon for mobile deployment leveraging the native Metal backend.

/bibliographystyleplain

/begintheibliography10

/bibitemvaswani2017attention Vaswani et al. ``Attention Is All You Need." NeurIPS 2017.

/bibitemchild2019generating Child et al. ``Generating Long Sequences with Sparse Transformers." 2019.

/bibitembeltagy2020longformer Beltagy et al. ``Longformer: The Long-Document Transformer." 2020.

/bibitemzaheer2020bigbird Zaheer et al. ``Big Bird: Transformers for Longer Sequences." NeurIPS 2020.

/bibitemliu2021swin Liu et al. ``Swin Transformer: Hierarchical Vision Transformer using Shifted Windows." ICCV 2021.

/bibitemdosovitskiy2020vit Dosovitskiy et al. ``An Image is Worth 16x16 Words." ICLR 2021.

/bibitembronstein2021geometric Bronstein et al. ``Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges." 2021.

/bibitemmcAleer2019deepcube McAleer et al. ``Solving the Rubik's Cube with Deep Reinforcement Learning and Search." Nature Machine Intelligence 2019.